

Анализ ситуации в компании

Компания оперирует ML-инфраструктурой, обслуживающей 85 продуктовых команд и около 600 инженеров, выполняющих 11000 задач ежедневно. Железо: 122 узла HGX H100/H200 в собственных дата-центрах и облачные мощности, включая узлы B200 и 150 карт A100. Суммарно инфраструктура насчитывает около 154 узлов класса HGX и дополнительные облачные ресурсы.

Нагрузка неоднородна: от задач рискованного скоринга, требующих до 800 ГБ оперативной памяти без гпу, до тяжелого обучения моделей, занимающего сервер целиком.

Система прошла путь от Slurm, который обеспечивал высокую утилизацию гпу в 50%, но ограничивал гибкость и мониторинг, к Kubernetes. Переход на стандартный Kubernetes изначально привел к деградации метрик (утилизация гпу упала до 20%). Текущее состояние характеризуется использованием решения Volcano, которое позволило поднять утилизацию гпу до 45% и внедрить механизм вытесняемых задач. Эксплуатация осложнена высокой частотой поломок оборудования (2 раза в неделю) и жестким регламентом обновлений, требующим вывода кластеров из продуктивного контура на 2 недели.

Ключевые задачи и цели внедрения платформы

- Переход от закупки серверов под конкретные команды к модели общего пула ресурсов для всех команд.
- Возможность быстро внедрять новые инструменты разработки и интегрировать платформу с внутренними бизнес-процессами.
- Повышение плотности размещения задач через механизм вытеснения.
- Получение детального мониторинга ресурсов и управления сетевым доступом, чего не хватало в Slurm.
- Внедрение гибких лимитов и гарантированных квот для разных классов задач (невывесняемые для критических расчетов, вытесняемые для массового обучения).
- Отказоустойчивость: Автоматический перезапуск задач при поломках оборудования, которые случаются дважды в неделю.

Ключевые проблемы внедрения

Основной трудностью стал конфликт между стандартным кюбером и спецификой ML-задач. Стоковый кюбер не умел адекватно работать с пакетной обработкой (batch processing): дедлайны задач считались с момента постановки в очередь, а не начала выполнения, отсутствовали механизмы вытеснения и гибкого квотирования. Это привело к огромному падению утилизации гпу с 50% до 20%.

Попытка написать собственный планировщик и оператор породила новые технические вызовы. Специалисты столкнулись с конфликтами между системными и кастомными компонентами, дублированием логики и низкой производительностью. Это вызывало простои кластеров и вынуждало команду инфраструктуры тратить ресурсы на переизобретение базовых функций оркестрации.

Процесс миграции и обновлений превратился в отдельную проблему для бизнеса. Регламент, требующий полного вывода кластера из эксплуатации на 2 недели для установки софта и еще на 2 недели для тестов, создавал огромные окна неэффективности. Каждое изменение архитектуры (происходящее раз в квартал) на месяц снижало продуктивность всех разработчиков на 10%.

Дополнительную сложность создала аппаратная нестабильность и специфика кода. Высокая частота поломок (2 раза в неделю) требовала массового внедрения вытесняемых задач. Однако написание кода, устойчивого к внезапным перезапускам с сохранением состояния, оказалось трудоемкой задачей для ML-инженеров.

Выбор Open-source решения

Компания выбрала шедулер Volcano, разработанный специально для запуска Cloud Native Batch-нагрузок в Kubernetes.

Выбор этого решения обусловлен тем, что стандартный планировщик Kubernetes ориентирован на запуск микросервисов, которые работают постоянно. Для ML-задач же важна пакетная обработка. Volcano внедряет

в экосистему кубера абстракции, как в Slurm: очереди, приоритеты и концепцию PodGroup. Это позволяет системе запускать либо все поды задачи, либо ни одного одновременно, что критично для распределенного обучения моделей, когда поды должны общаться друг с другом.

Внедрение Volcano позволило компании отказаться от поддержки собственного кастомного планировщика и перейти на стандартный для индустрии инструмент, сохранив при этом гибкость управления квотами и механизмами вытеснения задач.

Экономическая эффективность проекта

Для расчета возьмем за основу стоимость современного GPU-часа. По данным из презентации (по юнит экономике), при идеальной загрузке (порядка 70%) стоимость одного часа работы видеокарты B200 составляет примерно \$7,88. В эту сумму включены не только покупка железа, но и электричество, обслуживание в дата-центре и человеческие мощности.

Эффективность каждого этапа развития платформы напрямую зависела от того, насколько плотно удавалось утилизацию видеокарты:

- Slurm (утилизация 50%): Стоимость часа была довольно высокой — около \$11. Это было рабочее, но негибкое решение.
- Стандартный кюбер (утилизация 20%): Самый неудачный этап. Из-за плохой настройки инструментов карты простаивали 80% времени. Стоимость полезного часа подскочила выше \$25. Это в три раза дороже нормы.
- Внедрение Volcano (утилизация 45%): Переход на специализированное решение позволил снизить стоимость часа до \$12. Это все еще дороже идеала, но уже дешевле, чем просто арендовать аналогичные мощности в облаке (от \$18/час).

Серые зоны

Несмотря на рост технических метрик, в проекте остаются критические проблемы, которые расходуют бюджет:

1. Долгие обновления: Компания обновляет софт раз в квартал и тратит на это 4 недели. В это время кластер не приносит пользы. Получается, что 4 месяца в году (33% всего времени) сверхдорогое оборудование стоит выключенным. Кластер выводится из работы на 16 недель в году. При стоимости инфраструктуры в ~ \$38 млн и трехлетнем цикле амортизации, компания теряет около \$4.2 млн в год только на том, что железо простаивает во время настроек и тестов.
2. Потери человеко-часов: 600 разработчиков теряют 10% своей эффективности каждый раз, когда меняется инфраструктура. В масштабах компании это потеря огромного количества рабочих часов — эквивалентно, целая команда из 60 человек целый месяц в квартал занимается не обучением моделей, а борьбой с платформой. При средней стоимости специалиста для компании в 550 тыс. руб./мес, ежегодно теряется около 132 млн руб., ~ \$1.8 млн на том, что люди адаптируются к изменениям вместо обучения моделей.
3. Поломки: Оборудование выходит из строя дважды в неделю. Без специальных механизмов сохранения прогресса обучения каждый такой сбой заставляет начинать расчеты заново, что ведет к убыткам около \$0,7 млн в год только на потерянном времени работы видеокарт.

Варианты оптимизации процесса

Для повышения эффективности платформы без радикального увеличения числа сотрудников, можно предложить следующие методы:

- Бесшовные обновления (обновление узлов по частям). Это позволит избежать четырехнедельных простоев и сэкономить большую часть потерь по этому пункту.
- Дробление GPU для простых задач. Это может поднять общую утилизацию с 45% до ~ 60-70%, что снизит себестоимость GPU-часа почти в полтора раза.

- Внедрение инструментов автоматического сохранения чекпоинтов. Это позволит сократить время простоя после поломки узла с нескольких часов до 5-10 минут, минимизировав ежегодный убыток в \$0,7 млн.
- Упреждающий мониторинг выхода из строя видеокарт, настройка алертов на ранние признаки износа. Это позволит менять их до того, как они уронят длинную задачу обучения, требующую участия многих узлов.